

Instructions to Deploy your Project on a Server

1. Prepare your Back-End Project

- a. Create a `.env` file in your project's root directory. This file should include your database secrets (e.g., database name, username, password, host).
- b. Replace all hard-coded secrets in your code with variables from the `process.env` object. For example, if you have a database password, replace the hard-coded password with `process.env.DB_PASSWORD`.

```
const secret = {  
  user: process.env.USER_NAME,  
  password: process.env.PASSWORD,  
  host: process.env.HOST,  
  port: process.env.PORT,  
  database: process.env.DATABASE,  
  ssl: true  
};  
  
module.exports = secret;
```

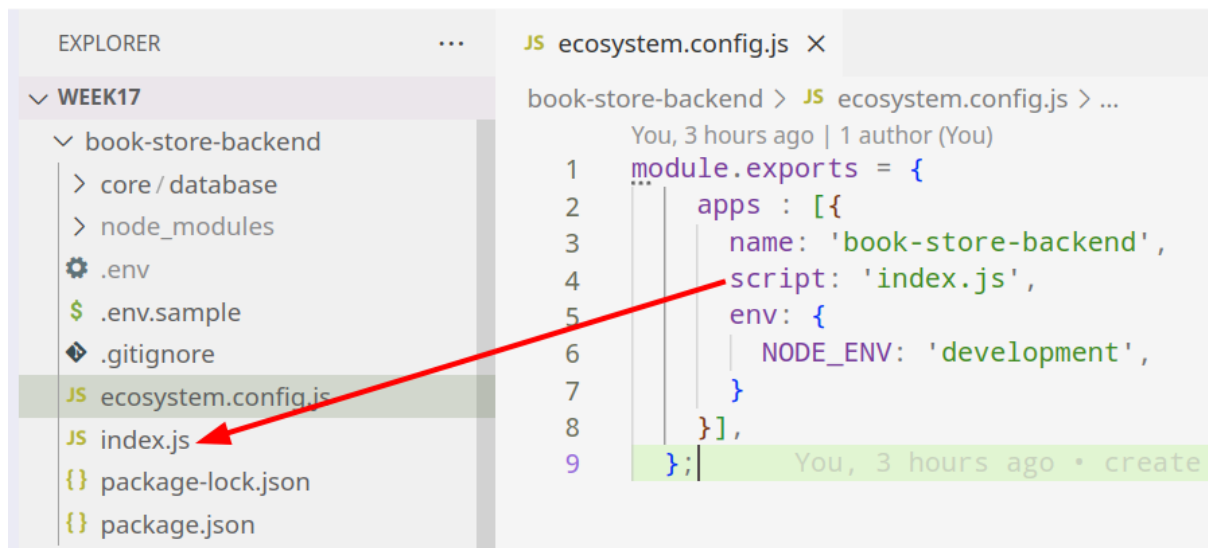
- c. Include the `.env` file in your `.gitignore` file to ensure it won't be committed to your Git repository. This is an important step for securing your secrets.

```
.gitignore ×  
book-store-backend > .gitignore  
You, 2 days ago | 1 author (You)  
1 /node_modules  
2 .env  
3
```

- d. Create a file named `ecosystem.config.js` in the root directory of your back-end project. This is a configuration file for PM2, a process manager for Node.js applications.
- e. The content of the file should be as follows:

```
module.exports = {  
  apps : [{  
    name: '<backend-app-name>',  
    script: '<your-entry-point-file-name>',  
    env: {  
      NODE_ENV: 'development',  
    }  
  }],  
};
```

Replace <backend-app-name> with the name of your back-end application, and <your-entry-point-file-name> with the name of the file that serves as the entry point to your back-end application (commonly index.js or app.js).



- f. As your front-end application uses port 3000, ensure that your back-end application is configured to use a different port, like 3001.
- g. Push your project to a GitHub repository.

2. Prepare your Front-End Project

- a. Create a .env file in your project's root directory. This file should include your backend base URL.
- b. Replace all hard-coded backend base URLs in your code with environment variables. For example:

```
const BASE_URL = import.meta.env.VITE_BACKEND_BASE_URL;
const { data } = await axios.get(`${BASE_URL}/books`);
```

- c. Include the .env file in your .gitignore file.
- d. Push your project to a GitHub repository.

3. Prepare your Arvan Cloud Server

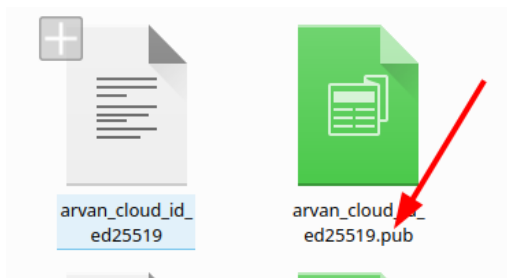
- a. Create an account on [Arvan](#) Cloud.
- b. You may need to charge your account in order to create an instance. It is recommended to charge at least 60,000T to avoid any issues during the creation process.
- c. Navigate to Server/Instances List and click on "New Instance".

- d. Choose "Iran - Tabriz" for your Data Center, "Ubuntu" for your OS, and "type: Sandbox, size: sb1 - small1" for your Instance Size.
- e. In the Advanced Settings section, select "SSH Key". In this step, you need to create a new SSH key on your system and share it with Arvan. Open your terminal (or PowerShell in Windows) and run the following command:

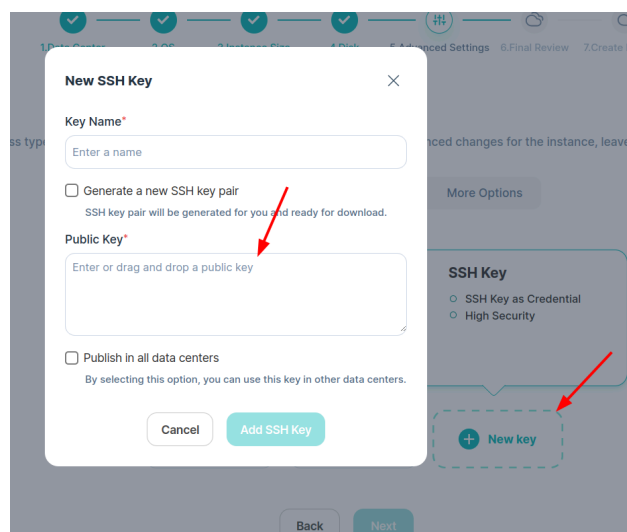
```
ssh-keygen -t ed25519 -C "Arvan_Cloud"
```

- f. The terminal will display the path where the key is being generated. Follow this path. Note: the .ssh folder may be hidden, so you may need to enable the display of hidden files in your file explorer.

```
ssh-keygen -t ed25519 -C "arvan_cloud"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/anahita/.ssh/id_ed25519): /home/anahita/.ssh/arvan_cloud_id_ed25519
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/anahita/.ssh/arvan_cloud_id_ed25519
Your public key has been saved in /home/anahita/.ssh/arvan_cloud_id_ed25519.pub
The key fingerprint is:
SHA256:7AWVU2RivhdtA4I1KkiEM6rq87/jeFoLheoPbCrma4c arvan_cloud
The key's randomart image is:
+--[ED25519 256]--+
|  oo  +*==       |
| +. . .+++ o    |
| . 0. . 0 ... +  |
```



- g. Open the file with the .pub extension and copy its content. Paste this content into the appropriate field in Arvan's interface. (Remember to always keep your SSH keys secure and never share them publicly.)



Instance: **ubuntu-sb1-small1-Shahriar-1**

Username: **ubuntu** SSH Key: **Anahita2**

ubuntu
1vCPU - 1 GB RAM - 15 GB Disk

Created

[View Instance Details <](#)

- h. Once the status of your server changes to "Created" in the Arvan Cloud Server interface, you will have access to your server IP and SSH access address. You can use these details to connect to your server and deploy your projects. (Click on "**View Instance Details**")

IPv4 Public Address: **188.121.98.247**

IPv4 Private Address: **-**

IPv6 Address: **-**

SSH Access: **ubuntu@188.121.98.247**

SSH key Name: **Anahita2**

4. Connect to your Server

- a. To connect to your server via SSH, use the following command:

```
ssh -i <path to your ssh key> <arvan ssh access address>
```

For example:

```
ssh -i ~/.ssh/id_ed25519_arvan ubuntu@5.34.199.203
```

Now you are in you server terminal.

5. Clone Your Projects on Server

To clone your projects from GitHub to your server, you'll need to generate an SSH key for GitHub:

- a. Generate a new SSH key with the following command:

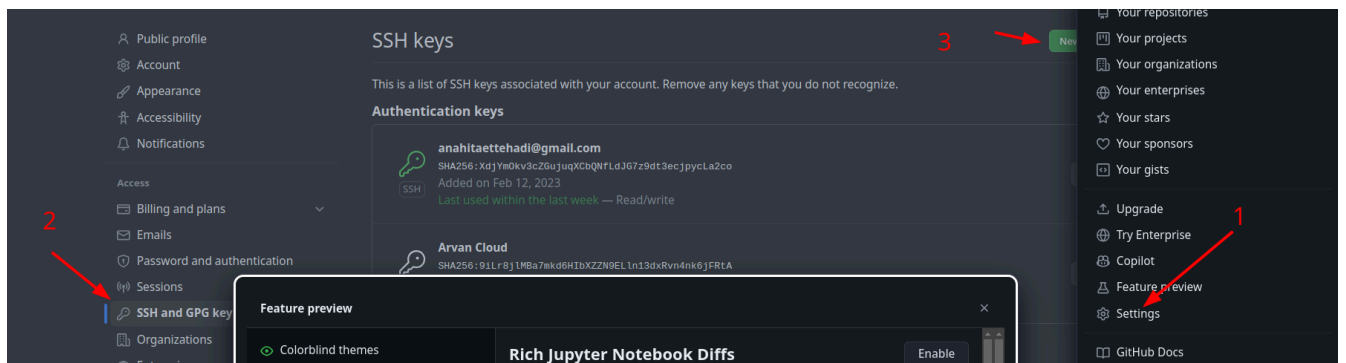
```
ssh-keygen -t ed25519 -C "github"
```

- b. Navigate to the .ssh directory and display the content of the public key file:

```
cd .ssh
cat id_ed25519.pub
cd ~
```

Copy the printed value from the terminal.

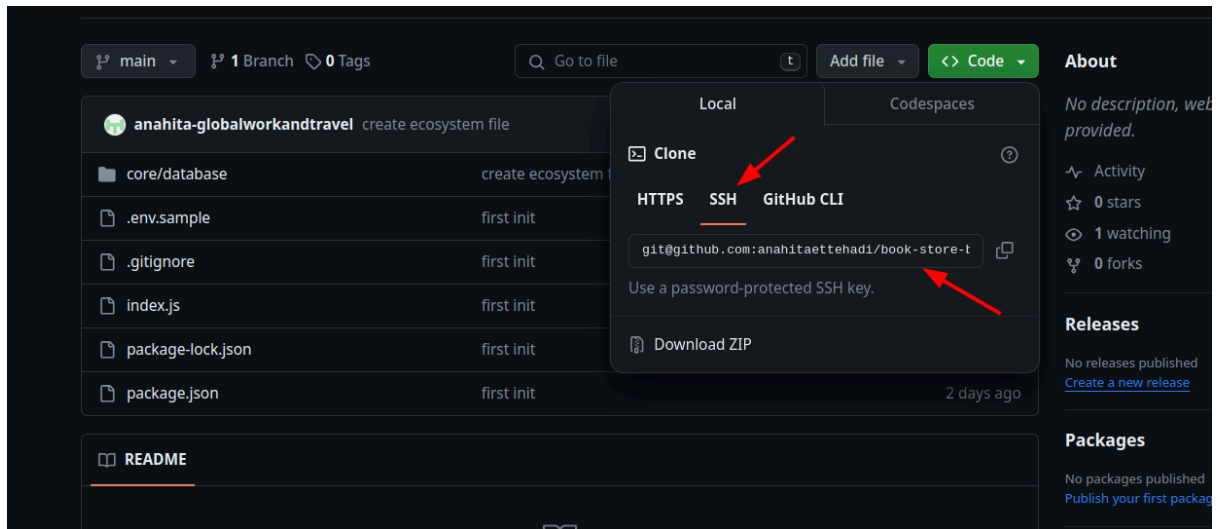
- c. Go to your GitHub account and navigate to Settings => SSH and GPG keys => New SSH key.
- d. Paste the copied value into the "Key" field and give it a title. This step will allow your server to communicate with your GitHub account securely.

A screenshot of the 'Add new SSH Key' form. The form has three main sections: 'Title', 'Key type', and 'Key'. The 'Title' field is empty. The 'Key type' dropdown is set to 'Authentication Key'. The 'Key' field contains a long string of characters. A red arrow points to the 'Key' field.

- e. Now that your server is set up to communicate with your GitHub account, you can clone your projects onto your server. Use the following command to clone each project:

```
git clone <project ssh URL>
```

Replace <project ssh URL> with the SSH URL of your project. You can find this URL on the main page of your project's GitHub repository.



f. Repeat this process for both your front-end and back-end projects.
Now, your projects are cloned and ready to be set up on your server.

6. Configure Your Server

a. Install node and NPM:

```
sudo apt-get update && sudo apt-get install -y ca-certificates curl
gnupg
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key |
sudo gpg --dearmor -o /etc/apt/keyrings/nodesource.gpg
NODE_MAJOR=20
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg]
https://deb.nodesource.com/node_$NODE_MAJOR.x nodistro main" | sudo tee
/etc/apt/sources.list.d/nodesource.list
sudo apt-get update && sudo apt-get install nodejs -y
```

b. Install PM2 and serve

```
sudo npm install -g pm2 serve
```

c. Install Caddy

```
sudo apt install -y debian-keyring debian-archive-keyring
apt-transport-https curl
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo
gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt'
| sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update
sudo apt install caddy
```

d. Install backend modules

```
cd /path/to/your/backend-project  
npm install
```

e. Create a .env file and fill it with the necessary environment variables:

```
nano .env  
cd ~
```

f. Install frontend modules

```
cd /path/to/your/frontend-project  
npm install
```

g. Create a .env file and fill it with the necessary environment variables and build the app:

```
nano .env  
npm run build  
cd ~
```

In your frontend project's .env file, set the value of VITE_BACKEND_BASE_URL to your Arvan server IP followed by the port number (81 in this case). For example:

```
VITE_BACKEND_BASE_URL=http://<arvan_server_ip>:81
```

Replace <arvan_server_ip> with the IP address that Arvan has provided for your server.

Remember to replace /path/to/your/backend-project and /path/to/your/frontend-project with the actual paths to your projects on the server.

7. Run Projects with PM2

a. Serve your frontend project with PM2:

```
pm2 serve /path/to/your/frontend-project/dist 3000 --name  
"<your-app-name>"
```

b. Navigate to your backend project directory and start your backend project with PM2:

```
cd /path/to/your/backend-project  
pm2 start ecosystem.config.js
```

Remember to replace /path/to/your/frontend-project/dist and /path/to/your/backend-project with the actual paths to your projects on the server, and <your-app-name> with the name of your application.

8. Configure Caddy File

- a. Open the Caddyfile:

```
sudo nano /etc/caddy/Caddyfile
```

- b. The content of the file should be as follows:

```
:80 {  
    reverse_proxy localhost:<react-app-port>  
}  
  
:81 {  
    reverse_proxy localhost:<express-app-port>  
}
```

Replace <react-app-port> with the port your React application is running on, and <express-app-port> with the port your Express app is running on. In this case, we are running the frontend on port 3000 and the Express app port is based on your app configurations.

- c. Restart Caddy to apply the changes:

```
sudo systemctl restart caddy
```

Now that your projects are deployed and running on your server, you should be able to access your website by entering the IP address that Arvan has provided into your web browser.